ZARI.ai Full Stack Technology & System Architecture Report

Frontend, Backend, Integration Layer, Vision Models, Parameters, RAG & LLM Engine

1. Frontend Technology Stack

The user interface of ZARI.ai is built using modern web development frameworks optimized for high performance, accessibility, and offline capability:

Core Framework: Next.js 14 (App Router)
UI Library: React 18 with TypeScript
Styling Engine: TailwindCSS v3 + Vanilla CSS Micro-animations
Icons & Motion: Lucide-React + Framer Motion (hydration-safe direct imports)
Static Export: Next.js Static Site Generation (output: 'export' -> frontend/out)
Image Handling: HTML5 FileReader API & Canvas for client-side image preview

2. Backend Technology Stack

The backend diagnostic microservice is built in Python, optimized for CUDA GPU acceleration and high concurrency:

Language & Runtime: Python 3.10+ / Uvicorn ASGI High-Performance Server
Web Framework: FastAPI (Asynchronous REST API Endpoints)
Deep Learning Framework: PyTorch 2.x + Torchvision (CUDA 12.x Accelerated)
Image Preprocessing: Pillow (PIL) + Torchvision Transforms (384x384 Tensor Scaling)
Log Monitoring: JSONL Production Monitor Log (backend/monitor_log.jsonl)

3. Single-Server Integration Architecture (Connecting Both)

To eliminate cross-origin complexity and allow deployment on a single port for the defense demonstration, the frontend static build is mounted directly inside FastAPI:

- Single Server Port: http://127.0.0.1:8000 (serves both Web UI and AI API)
- FastAPI Mount Code: app.mount('/', StaticFiles(directory='frontend/out', html=True))
- Diagnostic API Endpoint: POST /api/diagnose (Receives multipart image file + language code)
- Health Check Endpoint: GET /health (Returns model status, CUDA device, and SCRC threshold)
- Class Registry Endpoint: GET /api/classes (Returns 22 active production target classes)

4. Machine Learning Vision Model Suite & Parameters

ZARI.ai utilizes a 2-Stage Hierarchical Model Suite consisting of 4 PyTorch EfficientNetV2-B2 models:

Model Component	Architecture	Output Head	Exact Params	Checkpoint Path	Metric / Accuracy
Model A (Router)	EfficientNetV2-B2	3 Crop Classes	7,772,858	best_model_a_efficientnetv2_b2.	99.48% Accuracy
Model B (Tomato)	EDLEfficientNetB2	13 Diseases	7,786,948	best_model_b_tomato.pth	0.9787 Macro F1
Model B (Potato)	EDLEfficientNetB2	3 Diseases	7,772,858	best_model_b_potato.pth	0.9718 Macro F1
Model B (Pepper)	EDLEfficientNetB2	6 Diseases	7,777,085	best_model_b_pepper.pth	0.9963 Macro F1
TOTAL SYSTEM	4-Model Suite	22 Classes Total	31,109,749	355.73 MB Total Weights	99.1% Reliability

5. Evidential Deep Learning (EDL) & SCRC Safety Control

Dirichlet Softplus Evidence: e_k = Softplus(z_k) = ln(1 + exp(z_k))
Evidential Uncertainty Equation: u = K / S = K / sum(e_k + 1)
Calibrated Safety Threshold: tau_SCRC = 0.3175
Dual Safety Gate Action Rule: Accept if (u <= 0.3175 AND p_crop >= 0.80 AND p_disease >= 0.70) else REJECT

6. Vector RAG & LLM Advisory Engine

Vector Store Database:	ChromaDB v0.5+ HNSW Cosine Index (ml_pipeline/rag/chroma_db/)
Dense Embedder Model:	sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 (384d)
Knowledge Base Scope:	208 Chunks across 26 Records (22 Production Classes + 4 Quarantine Records)
Primary LLM Engine:	Meta Llama 3.1 8B Instant (8 billion params) on Groq LPUs (~750 tok/s)
Offline Fallback Synthesizer:	Deterministic Evidence Synthesis Engine (100% Offline Edge Mode)

7. Performance Benchmarks

Vision GPU Latency:	3.24 ms CUDA inference time
ChromaDB Retrieval Latency:	5.32 ms HNSW search time
Offline Synthesis Latency:	0.86 ms deterministic formatting time
TOTAL OFFLINE EDGE LATENCY:	9.40 ms (Vision + RAG + Offline Synthesis)
Groq Llama 3.1 8B API Call:	380 ms mean network + generation latency
TOTAL ONLINE CLOUD LATENCY:	389 ms (Vision + RAG + Groq API)